

B.Sc. Information Technology — Semester VI

AI-Driven Software Development

Course Code: 25UBIT607

UNIT 1

Introduction to AI in Software Engineering

Topics Covered:

- ① AI in Software Engineering — Introduction & Evolution
- ② AI-Assisted SDLC — Planning, Coding, Testing, Deployment
- ③ Tools Landscape — GitHub Copilot, TabNine, CodeWhisperer
 - ④ Prompt Engineering Basics for Code Generation
 - ⑤ LLMs & Transformer Architecture (GPT, Codex)

Exam Starts: 24th March | Credits: 4 | Max Marks: 150

1. Introduction to AI in Software Engineering

AI in Software Engineering means using intelligent tools to help developers write, test, deploy, and maintain software more efficiently. It helps developers work faster and more accurately throughout the entire software lifecycle.

1.1 What AI Does in Software Development

SDLC Phase	How AI Helps	Real-World Example
Writing Code	Auto-suggests or generates code from plain English	GitHub Copilot suggests login function
Testing	Creates test cases, detects security issues	AI tests empty input, wrong passwords
Deployment	Analyzes system, warns of performance issues	Netflix AI manages traffic spikes
Maintenance	Monitors logs, alerts on bugs, suggests fixes	AI alerts if many users fail to login

1.2 Real-Time AI Tools Examples

- GitHub Copilot — Suggests code while you type (like smart autocomplete)
- ChatGPT — Helps developers debug errors, explain code, generate programs
- SonarQube AI — Identifies bugs and security issues automatically

1.3 Why AI is Used in Software Engineering

- Saves time on repetitive coding tasks
- Reduces human errors and improves code quality
- Generates documentation automatically
- Detects missing validations and security flaws
- Makes development faster and more reliable

 *Student Example: AI can suggest login code, detect missing validations, and generate project documentation for college management system*

2. Evolution of Software Development with AI

Software development has evolved significantly from manual coding to intelligent AI-assisted development. This evolution is divided into three major stages.

Stage	Key Features	Example
1. Traditional Development	Manual coding Fixed requirements Time-consuming High dependency on skills	Developer writes complete Java program line by line in text editor
2. Agile & DevOps	Iterative development CI/CD Frequent updates User feedback driven	Mobile app receives weekly updates based on user reviews

Stage	Key Features	Example
3. AI-Powered Development	AI-assisted code generation Automated testing Natural language to code Higher speed	Prompt: 'Create REST API for student data' → AI generates code automatically

2.1 Key Changes Brought by AI

Change	Traditional Way	AI-Powered Way
Code Writing	Write every line manually	AI generates code from description
Error Detection	Manual debugging	AI warns about issues during coding
Feedback	After testing	Continuous real-time feedback
Skills Needed	Deep coding knowledge	Prompt writing + code review skills

✅ **Conclusion: AI has transformed developers from manual coders to problem solvers and system designers.**

3. AI-Assisted SDLC — Phase by Phase

The Software Development Life Cycle (SDLC) has been enhanced at every phase with AI tools that improve speed, accuracy, and quality.

3.1 Planning Phase

- Predicts project timelines based on past projects
 - Identifies risks early in the project
 - Converts user requirements into technical tasks using NLP
- 📌 *E-Commerce Example: AI analyzes past projects → predicts completion time | NLP converts 'User should track orders' into functional requirements*
- 📌 *Student Example: AI estimates how long a mini-project will take based on similar past projects*

3.2 Coding Phase

- Code auto-completion — suggests as you type
 - Function generation from plain English descriptions
 - Bug detection while coding in real-time
 - Understands comments and generates complete functions
- 📌 *Tools: GitHub Copilot, Amazon CodeWhisperer*
- 📌 *Example: Type 'Create a Python function to calculate student grade' → AI generates complete function*

3.3 Testing Phase

- Automatically generates test cases for all scenarios
- Predicts error-prone areas in the code
- Performs regression and visual testing

- ✦ *Banking App Example: AI tests money transfers under heavy load conditions*
- ✦ *Student Login Example: AI tests empty input, wrong password, SQL injection attempts*

3.4 Deployment Phase

- Predicts server failures before they happen
 - Optimizes cloud resources for cost and performance
 - Monitors application performance in real-time
- ✦ *Netflix Example: AI manages traffic spikes during popular show releases*
 - ✦ *Student Example: Deploying on AWS/Heroku — AI alerts if app crashes or slows down*

SDLC Phase	AI Tools Used	Key Benefit
Planning	NLP tools, AI estimators	Timeline prediction, risk detection
Coding	GitHub Copilot, CodeWhisperer, TabNine	Auto-completion, function generation
Testing	AI test generators, Selenium AI	Auto test cases, error prediction
Deployment	AWS AI, Google Cloud AI	Performance monitoring, failure prediction

4. Tools Landscape — GitHub Copilot, TabNine & CodeWhisperer

4.1 GitHub Copilot

GitHub Copilot is an AI coding assistant developed by GitHub + OpenAI. It helps developers write code faster by suggesting full lines or blocks of code based on context.

How it Works:

- Uses AI trained on millions of public code repositories
- Integrated into editors like Visual Studio Code
- Predicts code as you type — based on comments and surrounding code

Key Features:

- Smart code completions and whole function generation
- Context-aware suggestions based on variable names and comments
- Supports many languages — Python, JS, Go, C#, TypeScript, Java
- Can generate documentation, test cases, and API boilerplate

Real-Time Example:

You write a comment:

```
# Calculate factorial of a number
```

GitHub Copilot suggests:

```
def factorial(n):  
    if n == 0: return 1  
    result = 1  
    for i in range(1, n+1): result *= i  
    return result
```

📌 You can accept, modify, or reject the suggestion

4.2 TabNine

TabNine is an AI code completion tool that works with many code editors. It focuses on predictive code completion using machine learning models trained on code patterns.

Key Features:

- Completion suggestions based on patterns in your code
- Works offline in some configurations (local model)
- Custom training with your own codebase (Pro/Enterprise version)
- Lightweight with broad language and editor support

Real-Time Example:

If you start typing:

```
def calculate_total(price, quantity):
```

TabNine suggests:

```
    return price * quantity
```

Similarly for loops:

```
for i in range(
```

TabNine suggests:

```
for i in range(len(items)):
```

4.3 Amazon CodeWhisperer

Amazon CodeWhisperer is an AI development companion from AWS that offers code suggestions based on comments and existing code, with a strong focus on cloud and AWS services.

Key Features:

- Contextual suggestions with AWS-first focus
- Built-in security scanning — flags insecure code automatically
- Good for backend, serverless, and cloud applications
- Integrates with VS Code and JetBrains IDEs

Real-Time Example:

When writing AWS Lambda handlers or DynamoDB code, CodeWhisperer suggests correct AWS SDK usage and appropriate boilerplate code automatically.

4.4 Comparison Table — All Three Tools

Feature	GitHub Copilot	TabNine	Amazon CodeWhisperer
Primary Focus	AI-assisted code generation	Predictive code completion	Code suggestions + AWS context
Best For	Full function templates & logic	Fast local completions	Cloud-native & AWS applications
Context Awareness	Strong (comments & patterns)	Moderate (symbols & tokens)	Good (especially AWS code)
Security Guidance	Basic	Basic	Built-in insecure code detection
Offline Support	Limited	Yes (local model)	No (cloud-based)
Custom Training	GitHub repos only	Available (Pro)	Not typical
Editors Supported	VS Code, Neovim, JetBrains	All major editors	VS Code, JetBrains

4.5 When to Choose Which Tool?

Use This Tool	When You Need...
GitHub Copilot	Full code generation from comments Large public code corpus Natural language context
TabNine	Fast lightweight completions Local/offline work Private codebase training
Amazon CodeWhisperer	Building AWS cloud apps AWS best practices suggestions Security-aware code generation

5. Prompt Engineering Basics for Code Generation

Prompt engineering is the skill of writing clear, structured, and precise instructions (prompts) to an AI tool so that it generates accurate and useful code.

5.1 What a Prompt Tells AI

- What to build — the task or feature required
- How to build it — the approach or algorithm
- In which language — programming language or framework
- With what constraints — performance, security, simplicity

5.2 Why Prompt Engineering is Important

- Improves accuracy of AI-generated code
- Reduces errors and unnecessary output
- Saves significant development time
- Helps generate optimized and readable code
- Makes AI behave like a guided junior developer

5.3 General Prompt Template

```
Create <what> using <technology>.
Include <specific requirements>.
Return <output format>.
```

5.4 Key Prompt Engineering Rules with Examples

Rule	❌ Poor Prompt	✅ Good Prompt
Be Clear & Specific	Write code for login.	Create a login API using Node.js and Express. Use JWT. Return JSON only.
Mention Language	Write a program to sort numbers.	Write a Python program to sort a list using Quick Sort.
Break Complex Tasks	Create full e-commerce backend.	Step 1: MongoDB schema. Step 2: REST APIs. Step 3: Error handling.
Specify Input/Output	Write GCD function.	Create Java function. Input: two integers. Output: GCD. Use recursion.
Ask Best Practices	Create REST API.	Create REST API in Express. Follow best practices. Include error handling.

5.5 Using Comments for Code Generation

AI understands natural language comments very well — they act as inline prompts.

```
// Validate email format using regex
// Return true or false
```

AI generates:

```
function validateEmail(email) {
  const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```



```
return regex.test(email);  
}
```

5.6 Real-World Prompt Examples

Example 1 — REST API:

Create a REST API using Node.js and Express for managing student records. Include GET, POST, PUT, DELETE routes. Use MongoDB. Add error handling.

Example 2 — SQL Query:

Write SQL to find top 3 highest-paid employees from the employee table, ordered by salary.

Example 3 — React Component:

Create a React functional component for login form using useState and basic validation.

5.7 Iterative Prompting

Prompt engineering is iterative — you refine your prompts to get better results.

Round	Prompt	Improvement
First	Create a REST API.	Too vague — AI gives generic output
Refined	Create REST API using Express + MongoDB. Add validation and error handling.	Specific — AI gives production-ready code

5.8 Common Mistakes in Prompt Engineering

- ✗ Giving vague or incomplete instructions
- ✗ Not specifying programming language or framework
- ✗ Asking too many things in one prompt
- ✗ Not refining prompts when output is wrong
- ✗ Ignoring output constraints (format, comments, etc.)

Traditional Coding	Prompt Engineering
Write code manually	Guide AI to generate code
Time-consuming	Faster development
Error-prone	Reduced errors
Needs full logic knowledge	AI assists with logic

✅ **One-Line Definition:** Prompt engineering is designing clear, structured instructions to guide AI in generating accurate and efficient code.

6. LLMs & Transformer Architecture (GPT, Codex)

6.1 What are Large Language Models (LLMs)?

Large Language Models (LLMs) are AI systems trained on massive amounts of text data to understand, generate, and predict human language. They power most modern AI coding tools.

LLMs Can:

- Answer questions and explain concepts
- Generate code in multiple programming languages
- Translate between languages
- Summarize text and documents
- Assist in programming and problem-solving

LLM	Creator	Specialty
GPT (Generative Pre-trained Transformer)	OpenAI	General language understanding and generation
Codex	OpenAI	Specialized GPT for code generation
BERT	Google	Understanding context in text
LLaMA	Meta	Open-source language model

6.2 What is GPT?

GPT (Generative Pre-trained Transformer) is an LLM that is pre-trained on large datasets (books, articles, code, websites) using the Transformer architecture. It generates text one word (token) at a time.

📌 Example: Prompt 'Write Python to check prime numbers' → GPT generates correct readable Python code

6.3 What is Codex?

Codex is a specialized version of GPT that is fine-tuned specifically for programming tasks. It powers GitHub Copilot.

- Converts natural language → Code
- Understands multiple languages — Python, Java, C++, JS, SQL
- Understands code comments and docstrings

📌 Example: Comment '// Create REST API in Express' → Codex generates `app.get()` routes automatically

6.4 Transformer Architecture — The Backbone of LLMs

The Transformer is a deep learning architecture introduced in 2017 by Google in the paper 'Attention Is All You Need'. It is the foundation of GPT, Codex, and most modern AI tools.

Why Transformers are Better than older models (RNN/LSTM):

- Process data in parallel — much faster training
- Handle long-range dependencies — understands distant word relationships
- More scalable — can be trained on massive datasets

6.5 Key Components of Transformer Architecture

Component	What It Does	Example
Tokenization	Breaks text into tokens (words/subwords)	'I love AI' → ['I', 'love', 'AI']
Embedding Layer	Converts tokens into numerical vectors	'King' and 'Queen' have similar embeddings
Self-Attention	Focuses on important words, understands context	'it' in 'opened book because it was interesting' refers to 'book'
Multi-Head Attention	Multiple attention layers — each focuses on different relationships	One head: grammar Another: meaning Another: position
Feed Forward Network	Processes attention output, adds non-linearity	Improves language understanding
Positional Encoding	Adds word order information	'Dog bites man' ≠ 'Man bites dog'
Output Layer	Predicts next token using probability	'I am learning' → predicts 'AI' as next word

6.6 How GPT Works — Step by Step

Step	Process
1	User enters a prompt / question
2	Text is tokenized into small units
3	Tokens pass through multiple transformer layers
4	Self-attention analyzes context and relationships
5	Model predicts the next token (word/character)
6	Process repeats until full response is generated

6.7 Training of LLMs

3 Stages of Training:

- Pre-Training — Learns grammar, facts, patterns using unsupervised learning on massive data
- Fine-Tuning — Trained on specific tasks like coding, chat, Q&A
- RLHF (Reinforcement Learning from Human Feedback) — Humans rate outputs → improves safety and usefulness

6.8 GPT vs Codex — Comparison

Feature	GPT	Codex
Purpose	General language model	Code-specific model
Training Data	Text + some code	Heavy focus on code datasets

Feature	GPT	Codex
Primary Output	Text, answers, explanations	Code, functions, APIs, tests
Used In	ChatGPT	GitHub Copilot
Best At	Explaining concepts, summarizing	Generating working code

6.9 Advantages & Limitations of LLMs

✓ Advantages	✗ Limitations
High accuracy for language tasks	Requires large computational power
Understands context deeply	May produce incorrect (hallucinated) outputs
Handles large inputs	Lacks true understanding — pattern matching
Supports multiple languages	Dependent on quality of training data
Efficient after training	Potential bias from training data

✓ **One-Line Summary:** GPT and Codex are large language models built on Transformer architecture that use self-attention to understand context and generate human-like text and code.

Unit 1 — Quick Revision Summary

Topic	Key Points
AI in SE	Helps in writing, testing, deploying, maintaining software Saves time, reduces errors
Evolution	Traditional → Agile/DevOps → AI-Powered development
AI-Assisted SDLC	Planning(timeline) Coding(autocomplete) Testing(auto-cases) Deployment(monitors)
GitHub Copilot	AI code generation Based on OpenAI Comments → full functions VS Code
TabNine	Predictive completion Works offline All editors Custom training
CodeWhisperer	AWS-focused Security scanning built-in Cloud-native apps
Prompt Engineering	Clear + Specific + Language + Constraints = Good prompt Iterative refinement
Prompt Rules	Specify language Break complex tasks Use comments Ask best practices
LLMs	Trained on massive data Generate text/code GPT, Codex, BERT, LLaMA
GPT	General language model Pre-trained transformer Powers ChatGPT
Codex	Code-specialized GPT Powers GitHub Copilot Natural language → code
Transformer	Attention Is All You Need (2017) Parallel processing Better than RNN
Self-Attention	Focuses on important words Understands context and relationships
Training Stages	Pre-training → Fine-tuning → RLHF (human feedback)
Tokenization	Text broken into tokens → Embeddings → Vectors for processing

Important Exam Questions

- What is AI in Software Engineering? How does it help in SDLC? (5 marks)
- Explain the evolution of software development with AI (5 marks)
- Compare GitHub Copilot, TabNine, and Amazon CodeWhisperer (7 marks)
- What is Prompt Engineering? Explain rules with examples (5 marks)
- What are LLMs? Explain GPT and Codex with differences (5 marks)
- Explain the Transformer Architecture components with examples (7 marks)
- What is Self-Attention? How does it help LLMs understand context? (5 marks)
- What is RLHF? Explain the 3 stages of LLM training (5 marks)

✅ All the best! AISD Unit 1 — Introduction to AI in Software Engineering — Fully Covered! 🚀